

React JS State Hook

Sobre manutenção de
estados

Material

MOBILE_004



State Hook

Manutenção de estados





Agenda

- Componentes e estados
- State Hook
- Evento onClick
- Geração de números aleatórios



State Hook

- O State Hook, também conhecido como **useState**, é uma das funcionalidades do React que permite que componentes funcionais - uma forma mais simples e moderna de criar componentes no React - tenham seu próprio estado interno, algo que anteriormente era limitado a componentes de classe.



useState

- O useState é uma função que aceita um argumento, que é o estado inicial, e retorna um array com duas posições. A primeira posição é o valor atual do estado, e a segunda é uma função que pode ser usada para atualizá-lo.

Aqui está um exemplo básico de como useState é usado em um componente funcional do React:

```
import React, { useState } from 'react';

function Example() {
  // Declare uma nova variável de estado, que chamaremos de "count"
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Você clicou {count} vezes</p>
      <button onClick={() => setCount(count + 1)}>
        Clique aqui
      </button>
    </div>
  );
}
```

Evento onClick

- ▶ Ao clicar em um botão é comum que algo aconteça. Para chamarmos esta ação utilizamos o evento **onClick**.
- ▶ Neste exemplo, quando o usuário clicar no botão, o código associado ao evento **onClick** será executado.



por que o
evento **onClick**
é útil ?



Utilidade do onClick

- O evento **onClick** é uma ferramenta crucial na criação de páginas web e aplicações que respondem às ações dos usuários, melhorando a experiência do usuário e permitindo interações mais complexas.
- Essa característica permite que os desenvolvedores criem interfaces de usuário interativas e dinâmicas.
 - Por exemplo, um botão pode ter um evento **onClick** que aciona a submissão de um formulário, a abertura de uma nova página, a alteração de um estado dentro de um componente do React, entre outras ações.

E o State
Hook, por
quê é útil ?





Importância do State Hook

1. Gerenciamento de estado em componentes funcionais
2. Reatividade
3. Simplicidade e legibilidade
4. Modularidade

Gerenciamento de estado em componentes funcionais

- Antes da introdução dos Hooks, o gerenciamento de estado era restrito a componentes de classe. Com o **useState**, é possível ter estado local em componentes funcionais, o que torna o código mais legível e fácil de manter.

Reatividade

- O **useState** permite a criação de componentes reativos. Quando o estado de um componente é alterado utilizando a função de atualização fornecida pelo **useState**, o React sabe que precisa re-renderizar esse componente para refletir as alterações.

Simplicidade e legibilidade

- O **useState** torna o código mais limpo, pois elimina a necessidade de métodos de ciclo de vida complexos que eram necessários em componentes de classe para gerenciamento de estado.

Modularidade

- ▶ Com o **useState**, é possível definir múltiplos estados independentes dentro de um único componente. Cada chamada ao **useState** cria um novo estado local que pode ser gerenciado de forma independente, o que permite uma maior modularidade e reutilização de código.

Laboratório

- Exemplo de um número aleatório



Arquitetura do Exemplo

Vamos criar um componente funcional simples em React que gera um número aleatório entre 1 e 100 quando um botão é clicado.

Esses são os passos importantes

1. Importar React e useState
2. Criar o componente funcional
3. Inicializar o estado
4. Criar a função de geração de números aleatórios
5. Renderizar o componente

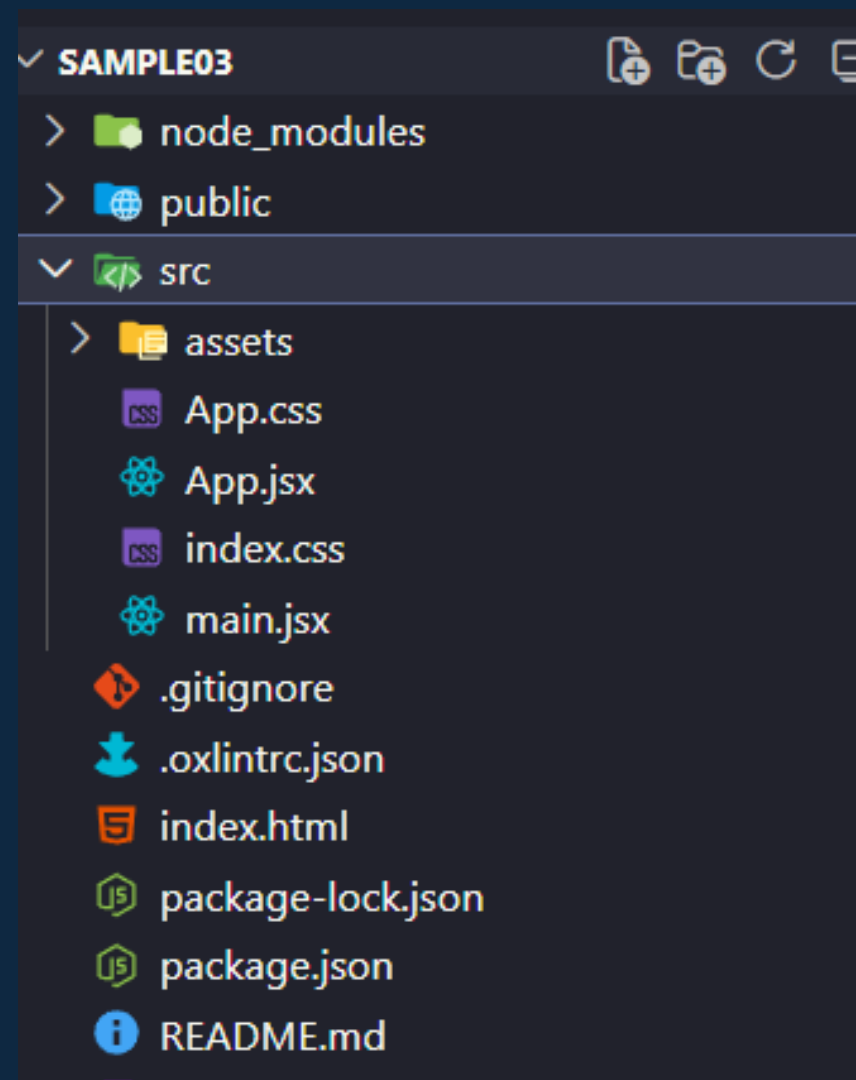


Passo 1 – Configuração do ambiente

1. Abra o terminal ou prompt de comando e navegue até o diretório onde deseja criar o projeto.
2. Execute o comando `npm create vite@latest sample03 -- --template react` para criar um novo projeto React chamado "myhello". Aguarde até que a criação do projeto seja concluída.
3. O processo anterior, de criação do projeto, pode levar alguns instantes enquanto é montado todo o boilerplate mínimo que uma aplicação React para a web precisa ter.
4. Entre no diretório [`cd sample03`] e digite [`code .`] para abrir o VS Code.

Passo 2 – Estrutura do projeto React

1. Abra o diretório do projeto criado no passo anterior.
[**cd sample03**]
2. Execute o comando **npm install**.



Passo 3: Preparando o componente pai

- Esse componente é chamado **App** e já foi criado junto com o projeto na aula anterior.
- O que precisamos fazer é remover o conteúdo que foi automaticamente criado dentro dele e inserir o nosso código.
- Nossa aplicação funcionará da seguinte forma:
 - O usuário clica no botão **Gerar número**;
 - Um número aleatório substitui o número 0.

Passo 4 – Altere o arquivo App.jsx



```
import './App.css'
```

```
export default function App() {  
  return (  
    <div className="App">  
  
    </div>  
  )  
}
```

Passo 5: Preparando Componente Home [Home.jsx]

- Com o nosso componente App criado, agora podemos começar a construir o único componente filho que vai compor nossa página. Componente Home, como mostra a Figura.

```
import { useState } from 'react'
import './App.css'

export default function Home() {
  const [numeroAleatorio, setNumeroAleatorio] = useState(0)

  function gerarNumero() {
    const numero = Math.floor(Math.random() * 100) + 1
    setNumeroAleatorio(numero)
  }

  return (
    <div className='conteudo-centralizado'>
      <h3>Número Aleatório</h3>
      <h1>{numeroAleatorio}</h1>
      <div className='area-botao'>
        <label>
          Clique no botão para gerar um número aleatório
        </label>
        <button onClick={gerarNumero}>Gerar</button>
      </div>
    </div>
  )
}
```



Estilizando os
componentes

STYLE·STYLE·STYLE·
STYLE·STYLE·STYLE·

Diretório src - Estilizando a página principal

Arquivo **App.css**:

```
● ● ●
.App{
  display: flex;
  justify-content: center;
  align-items: center;
  flex-direction: column;
  width: 100%;
  height: 100vh;
  background-color: #f8fafb;
}
.conteudo-centralizado{
  display: flex;
  justify-content: center;
  align-items: center;
  flex-direction: column;
  background-color: #fff;
  padding: 20px;
  box-shadow: 3px 6px 10px 6px rgba(0,0,0,0.05);
  box-sizing: border-box;
}
```

```
● ● ●
.area-botao{
  display: flex;
  justify-content: center;
  align-items: center;
  flex-direction: column;
  text-align: center;
}
button{
  margin-top: 20px;
  color: #ffffff;
  font-size: 12px;
  font-weight: 500;
  text-decoration: none;
  border: none;
  border-radius: 5px;
  background-color: #599465;
  box-shadow: 0px 3px 6px rgba(121, 121, 121, 0.1);
  padding: 10px 20px;
  outline: none;
  cursor: pointer;
}
```

Diretório src - Atualizando a página principal

Arquivo App.js:



```
import './App.css'
import Home from './Home'

export default function App() {
  return (
    <div className="App">
      <Home />
    </div>
  )
}
```

Resultado da Execução

Número Aleatório

0

Clique no botão para gerar um número aleatório

Gerar

Número Aleatório

31

Clique no botão para gerar um número aleatório

Gerar

Resultado

Este componente irá inicialmente exibir a mensagem "Clique no botão abaixo para gerar um número aleatório:". Quando o botão for clicado, um número aleatório entre 1 e 100 será gerado e exibido na tela, substituindo a mensagem inicial.

Biblioteca 'react-router- dom'

Rotas





Agenda

- Instalando a biblioteca 'react-router-dom'
- BrowserRouter
- Routes
- Route
- Link

Rotas

- Rotas, em desenvolvimento web, são essencialmente os diferentes URLs que uma aplicação web pode responder.
 - Por exemplo, você pode ter uma rota para a página inicial ('/'), uma rota para a página de contato ('/**contact**'), e assim por diante.
- As rotas são uma parte fundamental de qualquer aplicação web e definem a estrutura e a organização da aplicação.

Rotas no Endereçamento Web

- O conceito de roteamento em uma aplicação web tradicional é bastante direto: cada URL corresponde a um recurso específico no servidor, que pode ser uma página HTML, uma imagem, um arquivo CSS, entre outros.

Rotas em Aplicação SPA

- No entanto, com o advento das aplicações de página única (SPA - Single Page Applications), como as construídas com o React, o conceito de roteamento teve que ser repensado.
- Em uma SPA, todo o conteúdo da aplicação é carregado na primeira solicitação, e as subseqüentes mudanças de página são gerenciadas inteiramente pelo JavaScript no lado do cliente. Isso permite que as SPAs sejam muito mais responsivas do que as aplicações web tradicionais, já que não precisam recarregar a página inteira a cada solicitação.

Como usar a biblioteca react-router-dom

- No próximo exemplo a aplicação funcionará da seguinte forma:
 - O usuário seleciona uma das opções (**Front-end** ou **Back-end**);
 - A página (componente) referente à opção é renderizada na tela.

front-end

back-end

Renderização

- Esse resultado (sem recarregar a página) só é possível graças a biblioteca react-router-dom.
- Para chegarmos nesse resultado precisamos conhecer alguns componentes dessa biblioteca:
 - **BrowserRouter** => Componente que armazena as rotas;
 - **Routes** => Seleciona apenas um componente para exibir na tela baseado na url.
 - **Route** => Componente que associa a rota ao componente.
 - **Link** => Componente que permite navegar para outra página (componente).



Construindo nossa
aplicação

Mão na Massa

Laboratório

- ➔ Desenvolvimento de uma aplicação SPA usando o React JS. Nome da aplicação **sample04**.



Passo 1: Configuração do ambiente de desenvolvimento

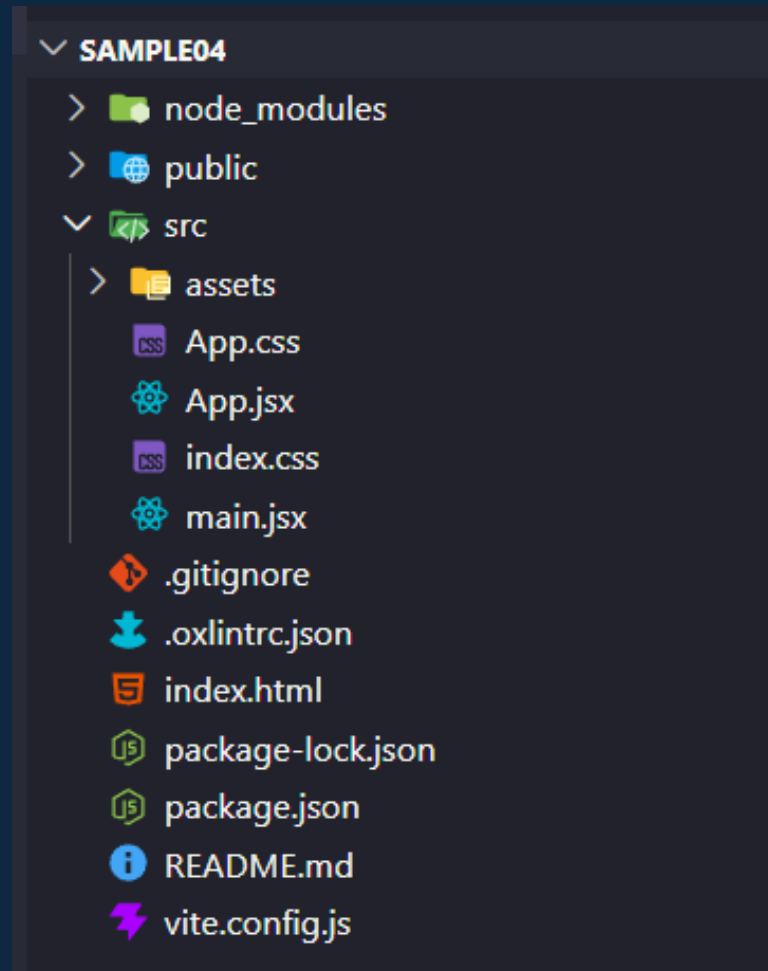
1. Execute o comando:

`npm create vite@latest sample04 -- --template react` para criar um novo projeto React chamado "sample04". Aguarde até que a criação do projeto seja concluída.

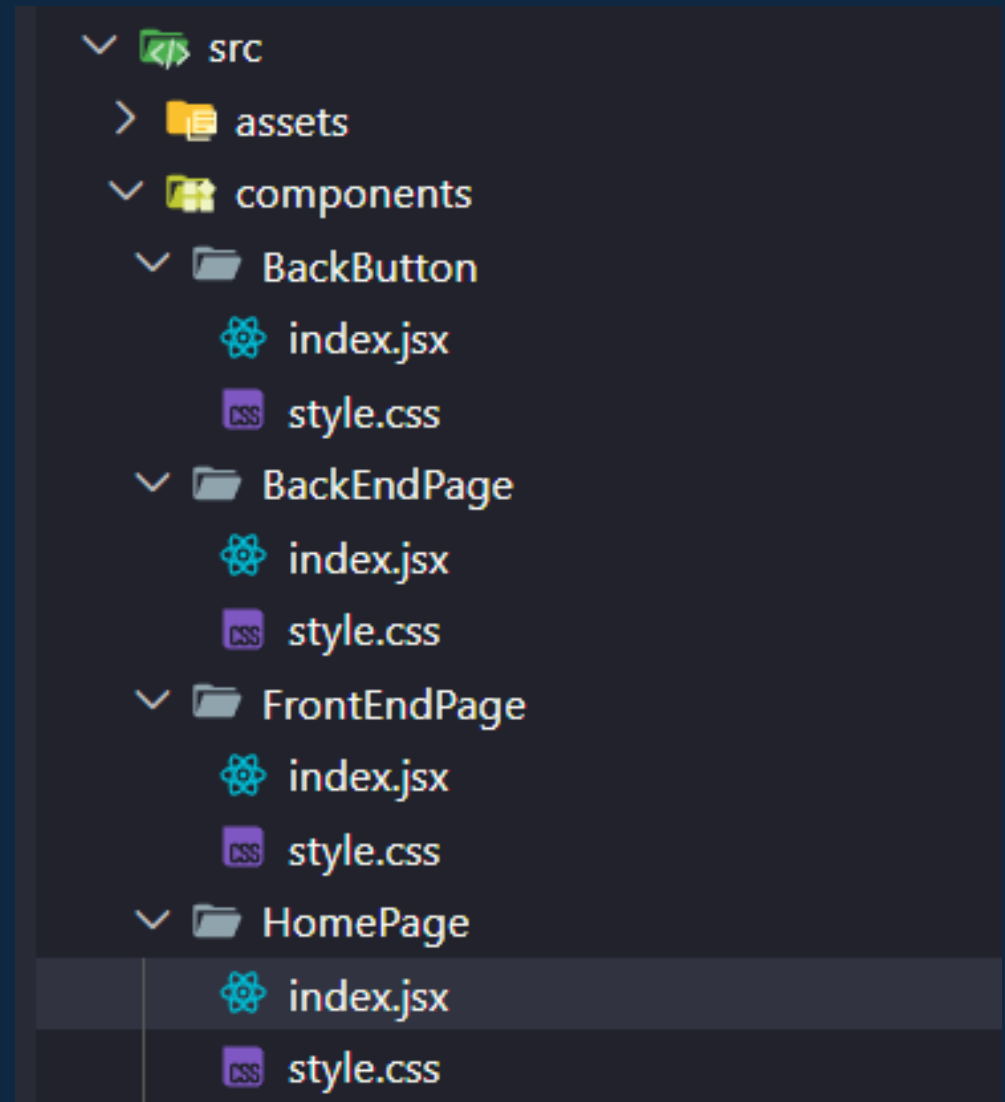
2. Entre no diretório [`cd sample04`] e digite [`npm install`]

3. Depois de instalado digite [`.code`] para abrir o VS Code.

Passo 2 – Estrutura do Projeto



Com os conhecimentos adquiridos nas aulas anteriores, defina a estrutura do projeto, como apresenta a figura:



Divisão da aplicação

Nossa aplicação está dividida em:

1. **App** - o componente principal;
2. **Routes** - quem guarda e gerência nossas rotas;
3. components divididos em:
 - A. **HomePage**- a página inicial;
 - B. **BackEndPage** - página exibida através da rota /back-end;
 - C. **FrontEndPage** - página exibida através da rota /front-end;
 - D. **BackButton** - componente utilizado para voltar para a página inicial. Esse componente é reutilizado em BackEndPage e FronEndPage.

Rotas do Projeto

- Rota "/"
 - Rota inicial, está associada ao componente HomePage
- Rota "/front-end"
 - Rota associada ao componente FrontEndPage
- Rota "/back-end"
 - Rota associada ao componente BackEndPage

Links

- Links que permitem a navegação:
 - / => endereço url padrão.
 - **Guia do React** => Link que abre uma url externa
 - **Botão Voltar** => Link que volta para a página inicial (HomePage)
 - **Botão Front-end** => Link que navega e exibe o componente (FrontEndPage)
 - **Botão Back-end** => Link que navega e exibe o componente (BackEndPage)

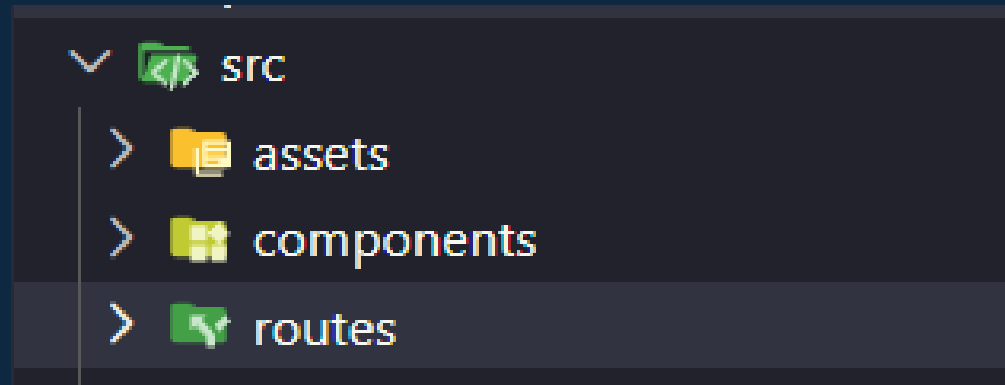
Passo 3: Preparando o ambiente para funcionamento das rotas

- O primeiro passo é instalar a biblioteca react-router-dom no nosso projeto.
- Instalando **react-router-dom**

```
PS C:\Users\memar\sample04> npm install react-router-dom  
:..
```

Passo 4: Configurando as rotas

- Dentro da pasta **routes**, crie o arquivo **index.jsx** para definirmos as rotas de nossa aplicação. Será a constante **Routes**.





```
import { BrowserRouter, Routes, Route } from "react-router-dom"

import HomePage from '../components/HomePage'
import FrontEndPage from "../components/FrontEndPage"
import BackEndPage from "../components/BackEndPage"

const MyRoutes = () => (
  <BrowserRouter>
    <Routes>
      <Route path="/" element={<HomePage />} />
      <Route path="/front-end" element={<FrontEndPage />} />
      <Route path="/back-end" element={<BackEndPage />} />
    </Routes>
  </BrowserRouter>
)
export default MyRoutes
```

Explicação a seguir

A constante **Routes** neste código é um componente funcional em React. O papel deste componente é definir as diferentes "rotas" ou "páginas" que a aplicação terá. Ele está utilizando várias funcionalidades oferecidas pela biblioteca **react-router-dom** para gerenciar essas rotas de maneira eficiente e flexível.

- **BrowserRouter:** Este é um componente de roteamento que usa a API de histórico HTML5 para manter sua interface de usuário em sincronia com a URL.
- **Routes:** Este é um componente que renderiza apenas a primeira rota **Route** ou **Redirect** que corresponde à localização atual.
- **Route:** Este é um componente que define uma rota. Ele tem duas props principais:
 - **path:** Define o caminho da URL para a rota.
 - **element:** Define o componente que será renderizado quando essa rota é acessada.

No exemplo, há três rotas definidas:

- Uma para o caminho **/**, que renderiza o componente **HomePage**.
- Uma para o caminho **/front-end**, que renderiza o componente **FrontEndPage**.
- E outra para o caminho **/back-end**, que renderiza o componente **BackEndPage**.

Passo 5: Importando as rotas para o App.js

- Nossas rotas estão criadas e agora precisamos colocá-las em nossa aplicação. Para isso abra o arquivo **App.js**



```
import './App.css'

import MyRoutes from './routes'

export default function App() {
  return (
    <div className="App">
      <MyRoutes />
    </div>
  )
}
```

Linkando HomePage as outras páginas

Componentes HomePage (index.jsx)

```
● ● ●  
  
import './style.css'  
import { Link } from 'react-router-dom'  
const HomePage = () => (  
  <div>  
    <h1>Escolha seu caminho</h1>  
    <p className="conteudo-opcoes">  
      <Link className='opcao-front-end' to="/front-end">Front-End</Link>  
      <Link className='opcao-back-end' to="/back-end">Back-End</Link>  
    </p>  
  </div>  
  
)  
export default HomePage
```

Nota

- Para concluirmos nossa SPA e finalizarmos a navegação da nossa SPA, precisamos permitir voltar para **HomePage** sem recarregar o navegador. Para isso vamos criar o componente **BackButton**.
- O componente **BackButton** foi codificado apenas uma vez e reutilizado nos componentes **FrontEndPage** e **BackEndPage**.

Componente: BackButton

Arquivo index.jsx



```
import './style.css'
import { Link } from 'react-router-dom'

const BackButton = () => (
  <div className='area-botao'>
    <Link className="botao-voltar" to="/">Voltar</Link>
  </div>
)
export default BackButton
```

Componente: BackEndPage

Arquivo: index.jsx



```
import './style.css'
import { Link } from 'react-router-dom'

const BackButton = () => (
  <div className='area-botao'>
    <Link className="botao-voltar" to="/">Voltar</Link>
  </div>
)
export default BackButton
```

Componente: FrontEndPage

Arquivo: index.jsx

```
import './style.css'
import BackButton from '../BackButton'
import { Link } from 'react-router-dom'

const FrontEndPage = () => (
  <div>
    <h1>React.js</h1>
    <p>
      O React.js (ou apenas React) é uma biblioteca JavaScript
      de código aberto desenvolvida pelo Facebook (Meta)
      usada para construir interfaces de usuário (UI),
      focada principalmente em aplicações de página única (SPAs).
    </p>
    <Link to="https://pt-br.reactjs.org/">React.js</Link>
    <BackButton />
  </div>
)
export default FrontEndPage
```



Estilizando os
componentes

STYLE·STYLE·STYLE·
STYLE·STYLE·STYLE·

Arquivos de Apoio. Estilos

- Arquivos que não serão comentados, apenas para melhorar a estética. As folhas de estilos.
- São eles:
 - **BackButton** -> style.css
 - **HomePage** -> style.css
 - App.css

BackButton – style.css



```
.area-botao {  
    width: 100%;  
    margin-top: 100px;  
}  
.botao-voltar {  
    display: inline-block;  
    position: relative;  
    padding: 20px;  
    text-decoration: none;  
    background-color: #f7df1e;  
    color: black;  
    font-weight: bold;  
    border: solid 1px rgba(0, 0, 0, 0.1);  
    border-radius: 12px;  
    box-shadow: 0 15px 25px -10px rgba(2, 8, 15, 0.5);  
}
```

HomePage - style.css

```
.conteudo-opcoes {
  display: flex;
  justify-content: space-between;
  margin-top: 50px;
}

.conteudo-opcoes a {
  padding: 20px;
  text-decoration: none;
  font-weight: 500;
}

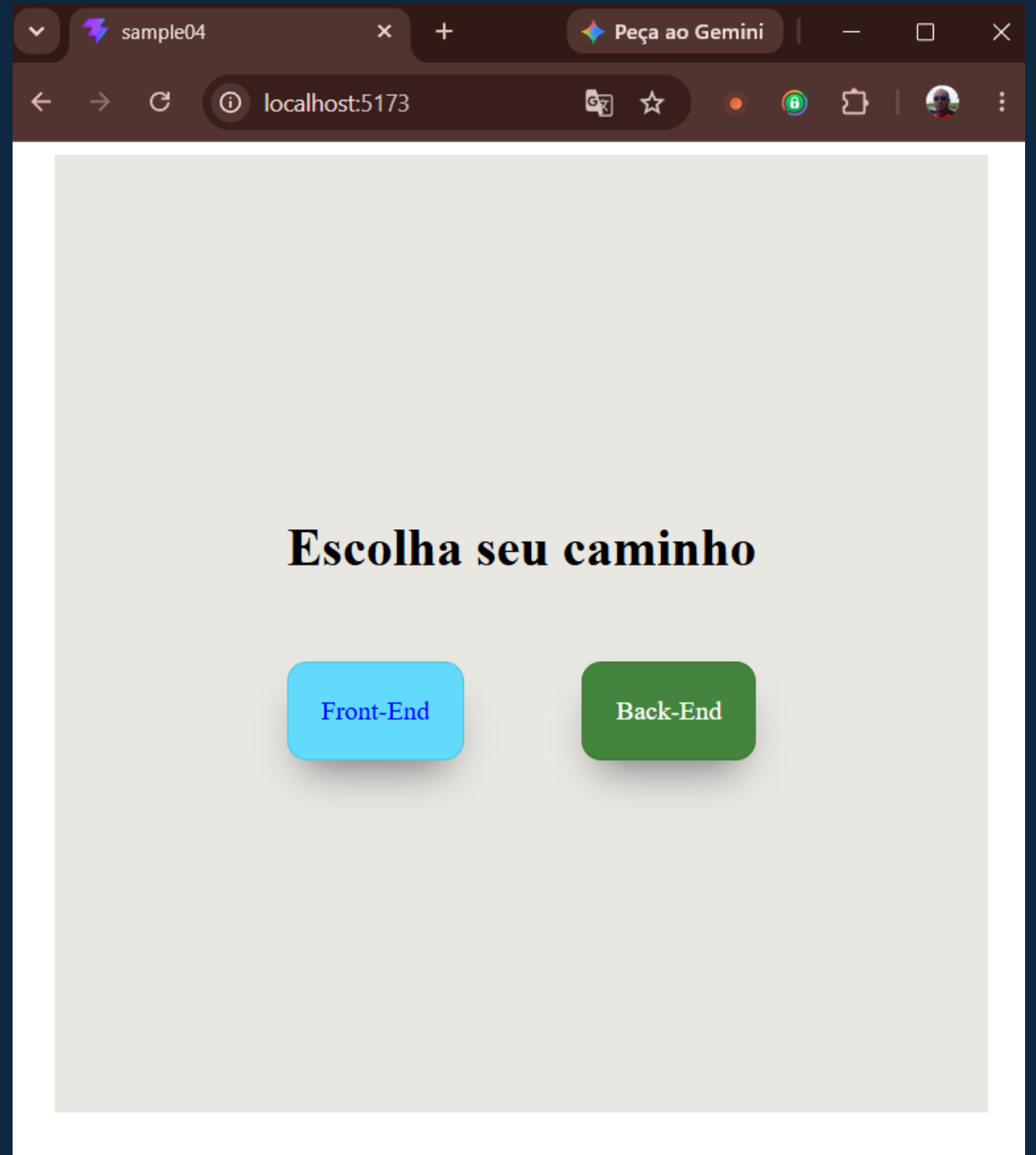
.opcao-front-end {
  background-color: #61dafb;
  color: blue;
  border: solid 1px rgba(0, 0, 0, 0.1);
  border-radius: 12px;
  box-shadow: 0 15px 25px -10px rgba(2, 8, 15, 0.5);
}

.opcao-back-end {
  background-color: #43853d;
  color: #fff;
  border: solid 1px rgba(0, 0, 0, 0.1);
  border-radius: 12px;
  box-shadow: 0 15px 25px -10px rgba(2, 8, 15, 0.5);
}
```

App.css

```
● ● ●  
.App {  
  margin: 0 auto;  
  text-align: center;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  width: 90vw;  
  height: 90vh;  
  padding: 20px;  
  box-sizing: border-box;  
  background-color: #e8e7e2;  
}
```

Resultado Final



Conclusão

➤ Estado e Controle

- Nesta aula, vimos que o controle de estados reativos transforma interfaces mudas em painéis interativos. Ao dominar o `useState`, você abre portas para manipular formulários, cronômetros, listas dinâmicas e contadores de forma leve.

➤ Aplicações Profissionais

- Mapear rotas utilizando o `react-router-dom` é o padrão profissional para converter simples páginas isoladas em grandes plataformas web integradas, com carregamento imediato e excelente experiência para o usuário final.



Agora
você será
capaz de:

- **Domínio da Reatividade no React:** Capacidade prática de implementar estados reativos com o Hook `useState` de maneira performática e organizada.
- **Manipulação de Ações com `onClick`:** Habilidade para controlar ações ativadas pelo clique do mouse e despachar modificações dinâmicas na tela.
- **Concepção Completa de SPAs:** Compreensão profunda da arquitetura de roteamento no client-side para evitar o recarregamento de páginas na web.
- **Gerenciamento Prático de Rotas:** Domínio no uso e configuração de componentes nativos como `BrowserRouter`, `Routes`, `Route` e `Link`.
- **Componentização Reutilizável de Rotas:** Criação e reutilização sistemática de componentes transversais, como botões de retorno (`BackButton`), reduzindo a duplicação de código.

Fim



Prof. Maromo